

Comprehensive Node.js App Setup with Swagger and Tests

This guide covers setting up a Node.js app using Express, integrating Swagger for API documentation, and writing test cases with Jest and Supertest. The instructions are broken down into clear steps to help you create a well-documented and tested API.

Technologies Covered:

- Node.js
- Express
- Jest
- Supertest
- Swagger (swagger-jsdoc and swagger-ui-express)

Step 1: Setup the Project

1. Create a new folder for your app and initialize a Node.js project:

```
```bash
mkdir node-test-app
cd node-test-app
npm init -y
```
```

2. Install the necessary dependencies for Express, Jest, Supertest, and Swagger:

```
```bash

npm install express

npm install --save-dev jest supertest

npm install swagger-jsdoc swagger-ui-express

```
```

Step 2: Create the Express App

Create a file named `app.js` in the root directory. This file sets up the basic Express server and integrates

Swagger for API documentation. The Swagger setup allows you to document the API directly in the code.

Create `app.js` with the following content:

```
```javascript

// app.js

const express = require('express');

const swaggerJsdoc = require('swagger-jsdoc');

const swaggerUi = require('swagger-ui-express');

const app = express();

// Swagger configuration

const swaggerOptions = {

 definition: {

 openapi: '3.0.0',
```

```
info: {
 title: 'Node Test API',
 version: '1.0.0',
 description: 'A simple Express API with Swagger documentation',
},
,

apis: ['./app.js'], // Points to the API documentation in the source code
};
```

```
// Initialize Swagger docs
```

```
const swaggerDocs = swaggerJsdoc(swaggerOptions);
```

```
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocs));
```

```
/**
```

```
 * @swagger
```

```
 * /:
```

```
 * get:
```

```
 * summary: Returns a Hello, World! message
```

```
 * responses:
```

```
 * 200:
```

```
 * description: Success
```

```
 * content:
```

```
 * application/json:
```

```
 * schema:
```

```
 * type: object
```

```
 * properties:
```

```
 * message:
```

```

* type: string
* example: Hello, World!
*/

app.get('/', (req, res) => {
 res.status(200).json({ message: 'Hello, World!' });
});

module.exports = app;
```

```

Step 3: Create the Server File

Create an entry point for starting the server by creating a file named `server.js`. This file will import the Express app and start the server.

Create `server.js` with the following content:

```

```javascript
// server.js

const app = require('./app');

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
 console.log(`Server is running on port ${PORT}`);
});
```

```

```
```
```

Start the server using the following command:

```
```bash
node server.js
```
```

## Step 4: Access the Swagger Documentation

Once the server is running, you can access the Swagger UI for the API documentation at:

```
```
http://localhost:3000/api-docs
```
```

Swagger UI provides an interactive interface to test and view the available API endpoints.

The `GET /`` route should display a "Hello, World!" message in the Swagger UI.

## Step 5: Write Unit and Integration Tests

Testing the API is crucial for ensuring reliability. Create a folder named `tests`` in the project root and

add a file named `app.test.js`` with the following content:

```
```javascript
// tests/app.test.js
```

```

const request = require('supertest');

const app = require('./app');

describe('GET /', () => {

  it('should return a 200 status and a Hello, World! message', async () => {

    const response = await request(app).get('/');

    expect(response.statusCode).toBe(200);

    expect(response.body).toHaveProperty('message', 'Hello, World!');

  });

});

describe('GET /api-docs', () => {

  it('should return a 200 status for the Swagger UI', async () => {

    const response = await request(app).get('/api-docs');

    expect(response.statusCode).toBe(200);

  });

});

...

```

This code uses Jest and Supertest to test two main aspects of the application:

- **GET /:** Confirms that the root route responds with a 200 status and the expected message.
- **GET /api-docs:** Confirms that the Swagger documentation page is accessible.

Run the tests using Jest:

```

```bash
npm test

```

'''

## Conclusion

Congratulations! You have successfully set up a Node.js app with Express, integrated Swagger for API documentation, and written comprehensive test cases using Jest and Supertest. This setup provides a solid foundation for building and documenting scalable APIs.

Feel free to expand this application by adding more routes and updating the Swagger documentation accordingly.